

BLM2002

Veri Yapıları ve Algoritmalar

Konu

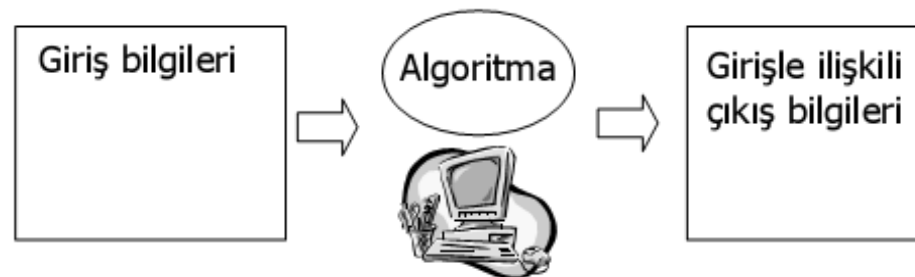
Algoritma Analizi ve Tasarımı

Hafta-02

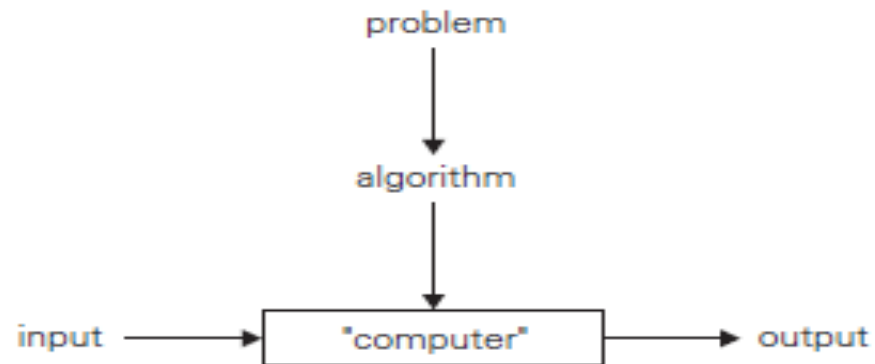


Algoritma Nedir?

- Algoritma
 - Bir problemin çözümü için geliştirilmiş özel metot
 - Girdileri çıktılarına dönüştüren sıralı hesaplama adımları
 - Tanımlanmış bir problemin çözümü için kullanılan araç
 - *"Bir problemin çözümü için izlenen sıralı ve anlaşılır buyruklar"*



Algoritma Hedefi



- Algoritma ile hedeflenen
 - Sonlu bir zaman içinde
 - Belirli girdiler ile
 - İstenilen çıktıyı elde etmektir.

Nasıl bir algoritma?

- Bir algoritma
 - Birden fazla biçimde sunulabilmeli
 - Net ve anlaşılır olmalıdır
 - Etkin ve faydalı olmalıdır
 - Sonlu veya sonlandırılabilir olmalıdır
 - Geliştirildiği problem için doğru olmalıdır

Algoritmanın Tarihçesi

- El Harezmi
 - 9. yyda yaşamış bir matematikçi
 - Algoritma ve cebir kavramlarının "atası" olarak bilinir
 - 0 sayısını ve x bilinmeyenini ilk kullanan kişi
- Euclid
 - En büyük ortak böleni bulma problemi için geliştirdiği çözüm ilk algoritmalarından biri olarak kabul ediliyor



EBOB Problemi

- Asal Çarpanlara Ayırma
 - EBOB'u bulunacak sayılar asal çarpanlarına ayrılır
 - Ortak asal çarpanların çarpımı EBOB'u verir
 - Adım 1 : m 'nin asal çarpanlarını bul
 - Adım 2: n 'nin asal çarpanlarını bul
 - Adım 3: Adım1 ve Adım2'de hesaplanan asal çarpanlardan ortak olanları belirle
 - Adım 4: Ortak asal çarpanları çarp

EBOB Problemi

- Asal çarpanlara ayırma algoritmasının işlem karmaşıklığı Euclid'in algoritmasından daha fazla
- Asal çarpanlara ayırma işleminin algoritması açık değil
- Eratosthenes'in Eleme Algoritması?

Asal Sayıları Bulma

- Eratosthenes'in Eleme Algoritması?
 - 2 ile n arasındaki asal sayıları bulmak için kullanılır
 - 1. iterasyonda 2'nin katları
 - 2. iterasyonda 3'ün katları
 - 3. iterasyonda 5'in katları elenir (?)

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
2	3		5		7		9		11		13		15		17		19		21		23		25
2	3		5		7				11		13				17		19				23		25
2	3		5		7				11		13				17		19				23		

Asal Sayıları Bulma

```
for  $p \leftarrow 2$  to  $n$  do  $A[p] \leftarrow p$ 
for  $p \leftarrow 2$  to  $\lfloor \sqrt{n} \rfloor$  do //see note before pseudocode
    if  $A[p] \neq 0$  //p hasn't been eliminated on previous passes
         $j \leftarrow p * p$ 
        while  $j \leq n$  do
             $A[j] \leftarrow 0$  //mark element as eliminated
             $j \leftarrow j + p$ 
//copy the remaining elements of  $A$  to array  $L$  of the primes
 $i \leftarrow 0$ 
for  $p \leftarrow 2$  to  $n$  do
    if  $A[p] \neq 0$ 
         $L[i] \leftarrow A[p]$ 
         $i \leftarrow i + 1$ 
return  $L$ 
```

EBOB Problemi

- Ardışık Tamsayı Kontrol Algoritması ile Çözüm (Consecutive integer checking algorithm)
 - EBOB (m,n) ikilisinin küçük olanından daha büyük olamaz
 - $\text{Min}(m,n)$ değerinin EBOB olup olmadığı kontrol edilir
 - $\text{Min}(m,n)$ EBOB ise işlem sonlanır,
 - » Değil ise bir azaltılarak tekrar kontrol edilir ($\text{min}(m,n)--$)
 - Algoritma
 - **Adım1:** $\min\{m, n\}$ değerini t' ye ata.
 - **Adım 2:** (m / t) işlemini yap, Eğer kalan = 0 ise Adım 3'e git;
 - Değil ise adım 4'e git
 - **Adım 3 :** (n/t) işlemini yap. Eğer kalan = 0 ise t değerini sonuç olarak döndür
 - Değil ise adım 4'e git
 - **Adım 4:** t değerini 1 azalt, Adım 2'ye git

EBOB Problemi

- Bu algoritma
 - Değerlerden biri 0 ise doğru çalışmaz (Neden?)
 - Algoritma girdilerinin kesin ve dikkatli bir şekilde belirlenmesi gerekliliği ve önemi

EBOB Problemi

- Euclid Çözümü

- $(m \bmod n)$ işleminin sonucu 0 olana kadar
 - $\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$ işlemini tekrar et
 - $\text{gcd}(60, 24) = \text{gcd}(24, 12) = \text{gcd}(12, 0) = 12$.
- Adım 1 : Eğer $n = 0$ ise m değerini sonuç olarak döndür ve dur
 - Değil ise 2. adıma git
- Adım2 : (m / n) işlemini yap, kalanı r 'ye ata.
- Adım 3 : n değerini m 'ye ata, r değerini n 'ye ata ($m = n, n = r$). Adım 1' dön.

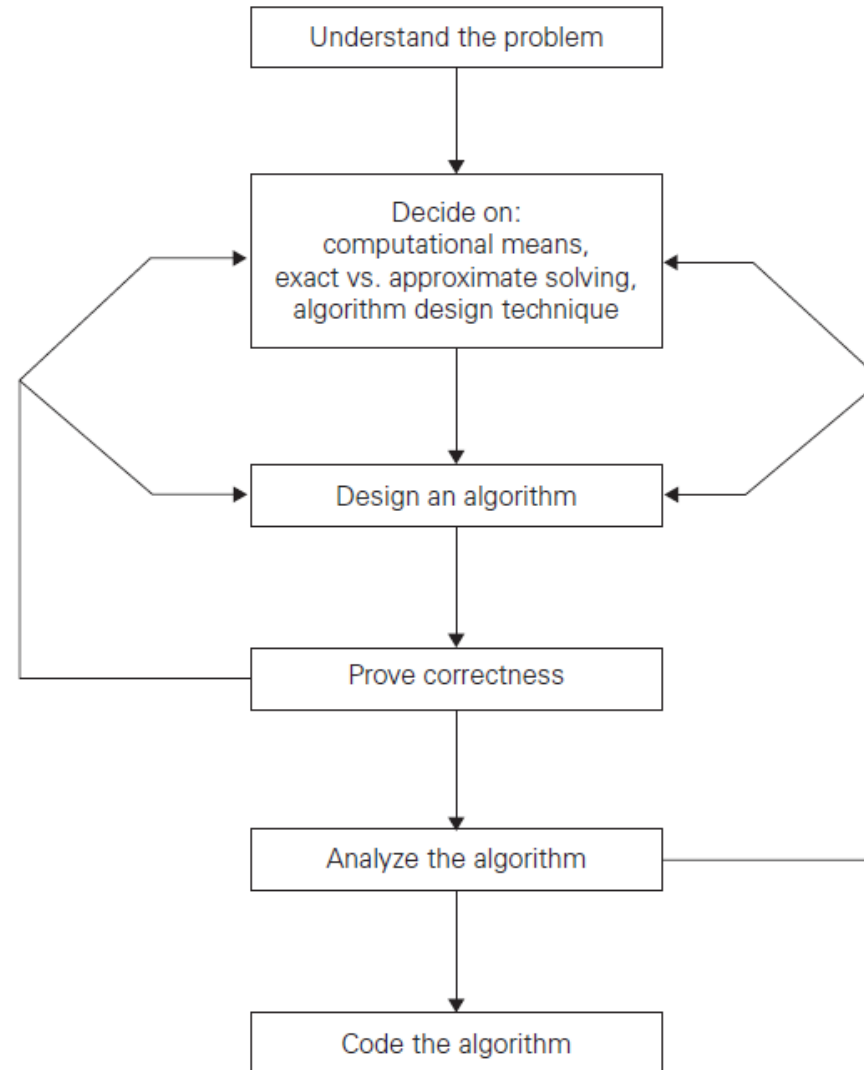
- Söзде Kod

```
//Computes gcd(m, n) by Euclid's algorithm
//Input: Two nonnegative, not-both-zero integers m and n
//Output: Greatest common divisor of m and n
while n ≠ 0 do
    r ← m mod n
    m ← n
    n ← r
return m
```

EBOB Problemi

- Euclid'in algoritmasının sonu var mı?
 - N değeri her iterasyonda küçülüyor.
 - 0'dan daha küçük, negatif, olamayacağı biliniyor.
 - İki pozitif sayının bölme işleminden negatif sayı veya kalan çıkamaz.
 - Er yada geç sıfıra ulaşarak algoritma duracaktır.

Algoritma Tasarım ve Analiz Süreci



Önemli Problem Türleri

- Sıralama
- Arama
- String işlemleri
- Graph problemleri
- Kombinasyonel Problemler
- Geometrik problemler
- Nümerik problemler

Önemli Problem Türleri

- Sıralama
 - Bir listedeki öğeleri artan sırada düzenlemek
 - Girdi : n adet sayıdan oluşan dizi $\langle a_1, a_2, \dots, a_n \rangle$
 - Çıktı : Girdinin $a'_1 \leq a'_2 \leq \dots \leq a'_n$ şeklinde yeniden düzenlenmesi
 - Neden Sıralama?
 - Arama işlemini kolaylaştırmak
 - Birçok algoritmanın altyordamı
 - Sıralama Anahtarı
 - Veri bütünüünün sıralamayı yönlendirmek için seçilmiş özel parçası
 - Bir başarı listesinin sıralanması (İsme, numaraya, nota göre)

Önemli Problem Türleri

- Örnek sıralama algoritmaları
 - Selection
 - Bubble sort
 - Insertion sort
 - Merge sort
 - Heap sort ...
- Sıralama algoritması karmaşıklığını değerlendirmek
 - Yapılan Karşılaştırma sayısı
- Sıralama algoritmaları için iki önemli özellik
 - Stabilite: İki eşit elemanın birbirlerine göre sıralanmadan önceki pozisyonlarında kalması
 - Bir dizide birbirine eşit iki eleman var.
 - Sıralamadan önceki konumları i ve j , $i < j$
 - Sıralamadan sonraki konumları i' ve j' ve $i' < j'$ ise algoritma stabil bir sıralama algoritmasıdır
 - Yerinde (In place) :
 - Fazladan bellek alanı gerektirmeyen sıralama algoritmaları
 - İstisna olarak küçük bellek birimleri kullanabilir
 - Dizinin tamamı kadar değil, bir değişken kadar

Önemli Problem Türleri

- Arama
 - Bir değeri verilen veri seti içinde bulma
- Sıralama algoritması örnekleri
 - Sıralı arama
 - İkili arama...
- Her durum için ideal arama algoritması yok
 - Hızlı fakat fazla bellek alanı
 - Sadece sıralı verilerde arama

Önemli Problem Türleri

```
Procedure ikiliarama(x:integer,  $a_1, a_2, \dots, a_n$ : artan tamsayılar)
i:=1; {i, arama aralığının sol bitiş noktasını gösterir}
j:=n; {j, arama aralığının sağ bitiş noktasını gösterir}
while i<j do
begin
    m:=[ (i+j)/2];
    if x>am then i:=m+1;
    else j := m;
end
if x=ai then konum:=i
else konum:=0;
{konum, x'e eşit olan terimin indisidir veya eğer x
    bulunamamış ise değeri sıfırdır}
```

Önemli Problem Türleri

- Sıralı dizi içerisinde 19 değeri aranıyor
- 1,2,3,5,6,7,8,10,12,13,15,16,18,19,20,22
 - 1,2,3,5,6,7,8,10
 - 12,13,15,16,18,19,20,22
 - 12,13,15,16
 - 18,19,20,22
 - 18,19
 - 20,22

Önemli Problem Türleri

- String İşleme
 - String
 - Bir alfabede yer alan karakterlerden oluşmuş dizi
 - Alfabetik, nümerik, özel karakterlerden oluşabilir
 - Örnek
 - Bir metin içerisinde www ifadesini aramak
 - @ karakterini aramak
 - Derleyiciler

Önemli Problem Türleri

- Graph Problemleri
 - Graph
 - Birbirine sınır adı verilen hatlarla bağlı noktalar grubu
 - Gerçek hayat problemleri
 - WWW modellemesi
 - Haberleşme ağları
 - Proje planlaması
 - Graph örnekleri
 - En kısa yol
 - Topolojik sıralama

Önemli Problem Türleri

- Kombinasyonel problemler
 - Çeşitli kısıtlılıkları sağlayan kombinasyonel çözümlerin bulunması
 - Maksimum değer, minimum maliyet
- Sorunlar
 - Probleme göre kombinasyonel çözümler çok büyüyebilir
 - Bu tip problemleri, kabul edilebilir bir sürede çözebilecek bilinen bir algoritma yok

Önemli Problem Türleri

- Geometrik problemler
 - Nokta, doğru, çokgen gibi geometrik nesneler ile ilgilenen formülleri
- Uygulama Alanları
 - Robotik
 - Bilgisayar Grafikleri
 - Tomografi
- En bilinen problemler
 - En yakın çift (Closest Pair)
 - Dışbükey gövde (Convex Hull)

En Yakın Çift Problemi

Verilen nokta kümesi içerisinde birbirine en yakın noktaları bulmak

NOKTA	X KOORDİNATI	Y KOORDİNATI
A	3	2
B	2	5
C	5	-3
D	-2	0
E	4	6
F	0	-4

İki Nokta Arası Mesafe : $\sqrt{(X1 - X2)^2 + (Y1 - Y2)^2}$

Önemli Problem Türleri

- Nümerik Problemler
 - Yaklaşık çözüme ulaşılmış problemlerin kesin sonuca ulaştırılmaya çalışılması
 - Denklem, denklem sistemi çözümleri
 - Kısıtlı integraller

Metin Arama

- Bir metin içerisinde aranan ifadeyi bulmak
 - Büyük / Küçük Harf duyarlılığı ?
 - Tam uyum / Ek almış ?

Algoritma Analizi

- Algoritma analizi, tasarlanan program veya fonksiyonun belirli bir işleme göre matematiksel ifadesini bulmaya dayanır.
- Burada temel hesap birimi seçilir ve programın görevini yerine getirebilmesi için bu işlemde kaç adet yapılması gerektiğini bulmaya yarayan bir bağıntı hesaplanır.
- Eğer bu bağıntı zamanla ilgiliyse çalışma hızını, bellek gereksinimiyle ilgiliyse bellek gereksinimini ortaya koyar.

Algoritma Analizi Çerçevesi

- Algoritma Analizinde Göz Önünde Bulundurulması Gerekenler Neler?
 - Algoritmanın Doğruluğu (Correctness)
 - Zaman Verimliliği (Time Efficiency)
 - Bellek Alanı Verimliliği (Space Efficiency)
 - Gelişen donanım teknolojisi ile artık çok önemli değil
 - Uygunluk, en iyilik (Optimality)

Algoritma Analizi Çerçevesi

- Girdi Büyüklüğünün Ölçülmesi
- Çalışma Zamanı Ölçü Biriminin Belirlenmesi
- Büyüme Derecesi (Order of Growth)
- En kötü durum, en iyi durum, ortalama durum değerlendirmeleri

Girdi Büyüklüğünün Ölçülmesi

- Girdi Büyüklüğünün Ölçülmesi
 - Tüm algoritmalar için büyük girdiler üzerinde çalışma süresi daha uzundur
 - Büyük dizi içinde arama
 - Büyük boyutlu matrisleri çarpma
 - Algoritmaya girilen veriyi belirten n değerinin belirlenmesi önemli
 - Bazı algoritmalar için kolay
 - Arama, sıralama, eleman sayısı bulma
 - Bazı algoritmalar için değil
 - İki matrisin çarpımı
 - » Matrislerin derecesi mi? Eleman Sayıları mı?
 - Yazım hatası programı
 - » Karakter Sayısı mı? Kelime Sayısı mı?

Algoritma Performansı

- Performans değerlendirme metrikleri:
 - Gerekli bellek alanı
 - Çalışma süresi
- Çalışma süresi en çok kullanılan standarttır.
 - Ölçülebilir ve karşılaştırması kolay
 - Genellikle performans, girdi sayısının bir fonksiyonudur:
 - Örnek: Veri elemanı sayısı

Çalışma Süresi Karmaşıklığının Ölçülmesi

- Genellikle çalışma süresi tahmini, tam olarak zaman birimi cinsinden yapılmaz.
(örn. Bir algoritmanın toplam kaç mili saniyede tamamlanacağı)
- Bunun yerine, bir algoritmanın hızının problemin girdi sayısına göre değişimi tahmin edilir.
(örn. İşlem sayısı).
- Problemin boyutuna (N) bağlı olan bir $f(N)$ fonksiyonu tanımlanır, bu fonksiyon algoritmanın çalışma süresi ile doğru orantılıdır.

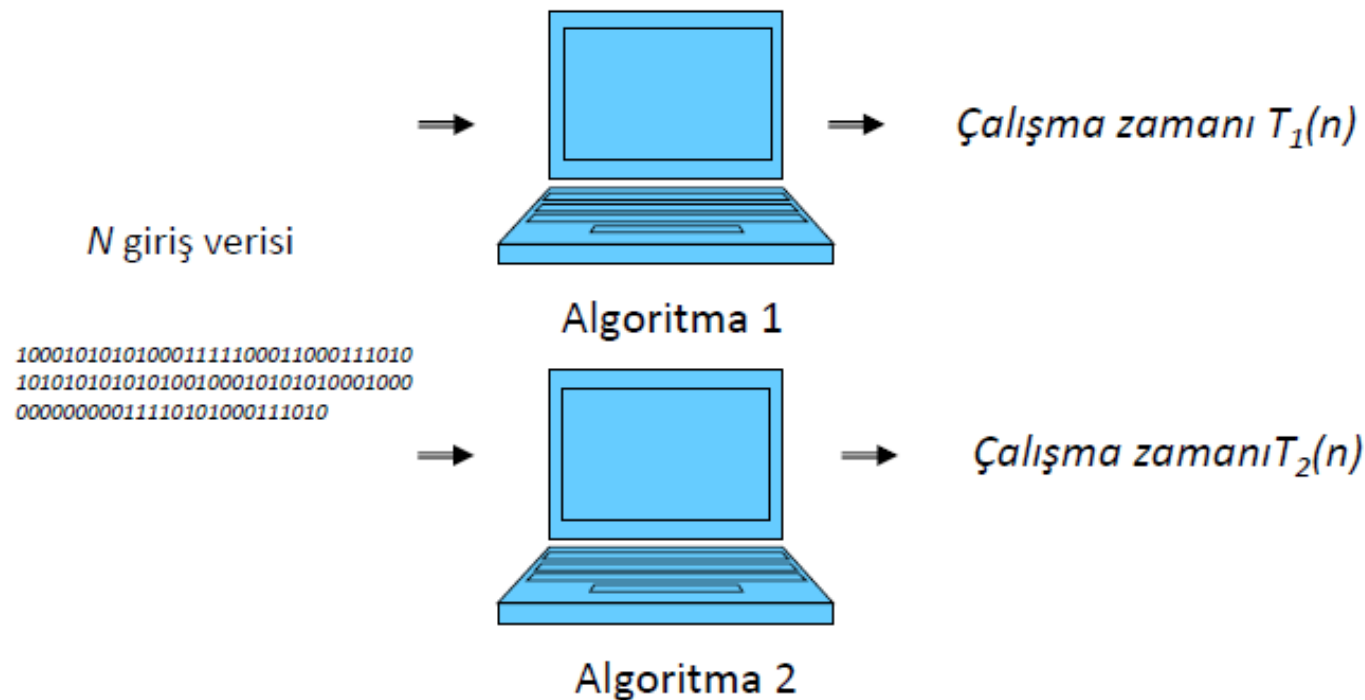
Yürütme Zamanı Analizi

- Yürütme Zamanı;
- Bir programın veya algoritmanın işlevini yerine getirebilmesi için, temel kabul edilen işlevlerden kaç adet yürütülmesini veren bir bağıntıdır ve $T(n)$ ile gösterilir.
- Temel hesap birimi olarak, programlama dilindeki deyimler seçilebildiği gibi döngü sayısı, toplama işlemi sayısı, atama sayısı, dosyaya erişme sayısı gibi işler de temel hesap birimi olarak seçilebilir.

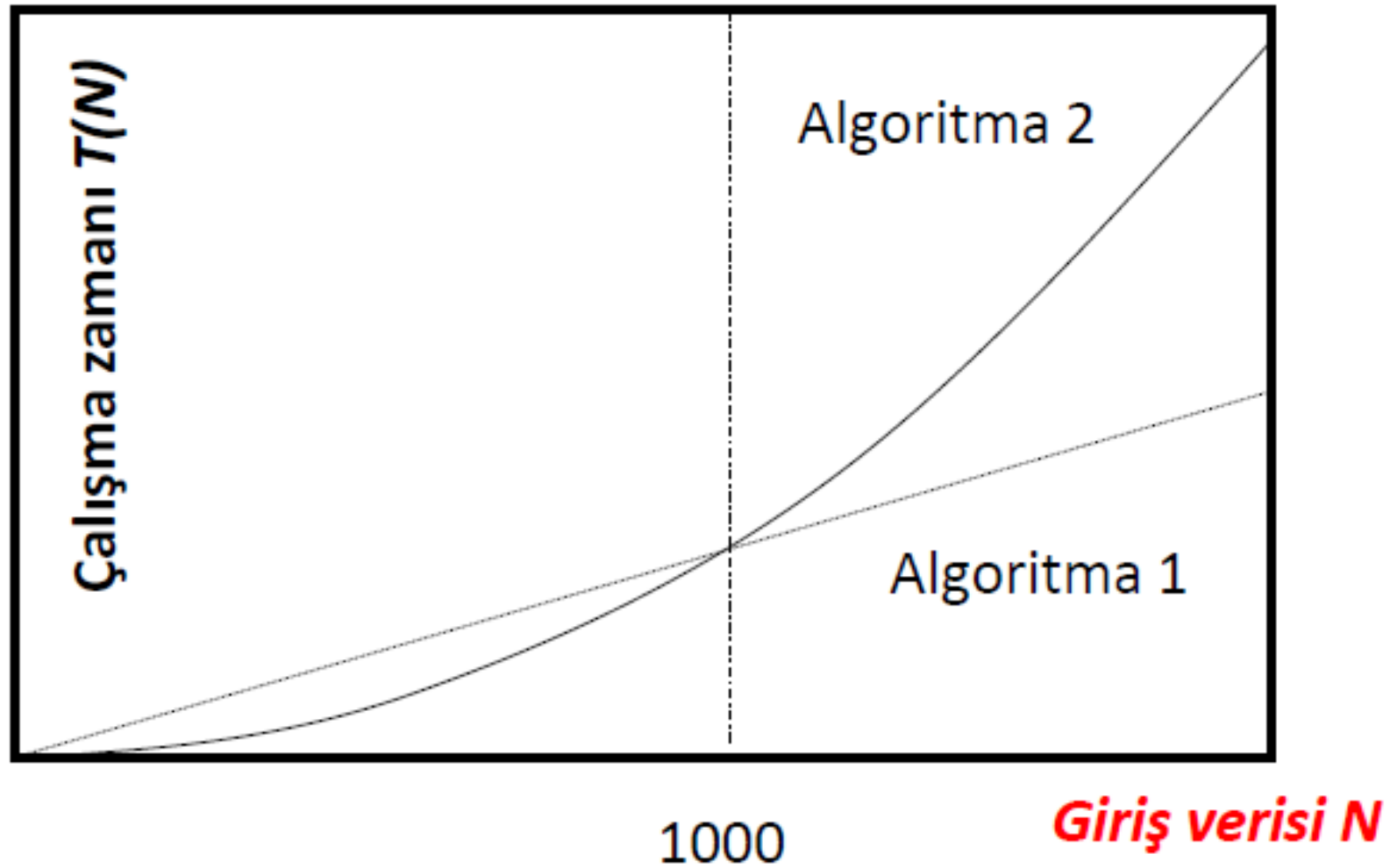
Yürütme Zamanı Analizi

Algoritma 1, $T_1(N)=1000N$

Algoritma 2, $T_2(N)=N^2$



Yürütme Zamanı Analizi



Çalışma Zamanlarının Özeti

N	T1	T2
10	10^{-2} sec	10^{-4} sec
100	10^{-1} sec	10^{-2} sec
1000	1 sec	1 sec
10000	10 sec	100 sec
100000	100 sec	10000 sec

N değerinin 1000'den küçük olduğu durumlarda iki algoritma arasındaki çalışma zamanı ihmal edilebilir büyüklüktedir.

Analiz

Çalışma zamanının kesin olarak belirlenmesi zordur

- Giriş verilerine bağlı olan en iyi durum (**best case**)
- Ortalama durum (**average case**); hesaplanması zordur
- Diğerlerine göre en kötü durum (**worst case**); hesaplanması kolaydır

Bunun için çeşitli notasyonlardan faydalanılır.

Aritmetik Ortalama için $T(n)$ Hesabı

Örnek 1: Aşağıda bir dizinin aritmetik ortalamasını bulan ve sonucu çağırana gönderen **bulOrta()** fonksiyonun kodu verilmiştir. Bu fonksiyonun yürütme zamanını gösteren **$T(n)$** bağıntısını ayırık C dili deyimlerine göre belirleyiniz.

```
float bulOrta(float A[], int n) {  
    {  
        float ortalama, toplam=0;  
        int k ;  
1-    for(k=0;k<n;k++)  
2-        toplam+=A[k];  
3-    ortalama=toplam/n  
4-    return ortalama;  
    }
```

Aritmetik Ortalama için T(n) Hesabı

Temel Hesap Birimi	Birim Zaman (Unit Time)	Frekans(Tekrar) (Frequency)	Toplam (Total)
float bulOrta(float A[], int n)	-	-	-
{	-	-	-
float ortalama, toplam=0;	-	-	-
int k ;	-	-	-
1- for(k=0;k<n;k++)	1,1,1	1, (n+1), n	2n+2
2- toplam+=A[k];	1	n	n
3- ortalama=toplam/n	1	1	1
4- return ortalama;	1	1	1
}	-	-	-
	T(n)=3n+4		

En Küçük Eleman Bulma için $T(n)$ Hesabı

Örnek 2: Aşağıda bir dizi içerisindeki en küçük elemanı bulan **bulEnKucuk()** adlı bir fonksiyonun kodu görülmektedir. Bu fonksiyonun yürütme zamanını gösteren $T(n)$ bağıntısı ayrık C dili deyimlerine göre belirleyiniz.

```
float bulEnKucuk(float A[])
{
    float enkucuk;
    int k ;
1-   enkucuk=A[0];
2-   for(k=1;k<n;k++)
3-       if (A[k]<enkucuk)
4-           enkucuk=A[k];
5-   return enkucuk;
}
```

En Küçük Eleman Bulma için T(n) Hesabı

Temel Hesap Birimi	Birim Zaman (Unit Time)	Frekans(Tekrar) (Frequency)	Toplam (Total)
float bulEnKucuk(float A[]) {	-	-	-
float enkucuk;	-	-	-
int k ;	-	-	-
1- enkucuk=A[0];	1	1	1
2- for(k=1;k<n;k++)	1,1,1	1, n, (n-1)	2n
3- if (A[k]<enkucuk)	1	n-1	n-1
4- enkucuk=A[k];	1	n-1	n-1
5- return enkucuk;	1	1	1
}	-	-	-
			T(n)=4n

Aşağıdaki C fonksiyonu için toplam işlem sayısını hesaplayalım

```
1 float Avg(int A[], int N)
2 {
3   float ort, tot=0;
4   int i;
5   for (i=0; i < N; i++)
6     tot = tot + A[i];
7
8   if (N > 0)
9     ort = (float)tot/N;
10
11   return ort;
12 }
```

Satır	İşlem	Maliyet	Tekrarlar
1	tot=0	1	1
2	i=0	1	1
2	i < N	1	N+1
2	i++	1	N
3	tot = tot + A[i]	1	N
4	N > 0	1	1
5	(float) tot	1	1
5	... / N	1	1
6	return ort	1	1
Toplam T(N) =			3N+7

Karmaşıklık (Complexity)

Karmaşıklık; bir algoritmanın çok sayıda parametre karşısındaki değişimini gösteren ifadelerdir. Çalışma (Yürütme) zamanını daha doğru bir şekilde bulmak için kullanılırlar.

Genel olarak, az sayıda parametreler için karmaşıklıkla ilgilenilmez; eleman sayısı n 'nin sonsuza gitmesi durumunda algoritmanın maliyet hesabının davranışını görmek veya diğer benzer işleri yapan algoritmalarla karşılaştırmak için kullanılır.

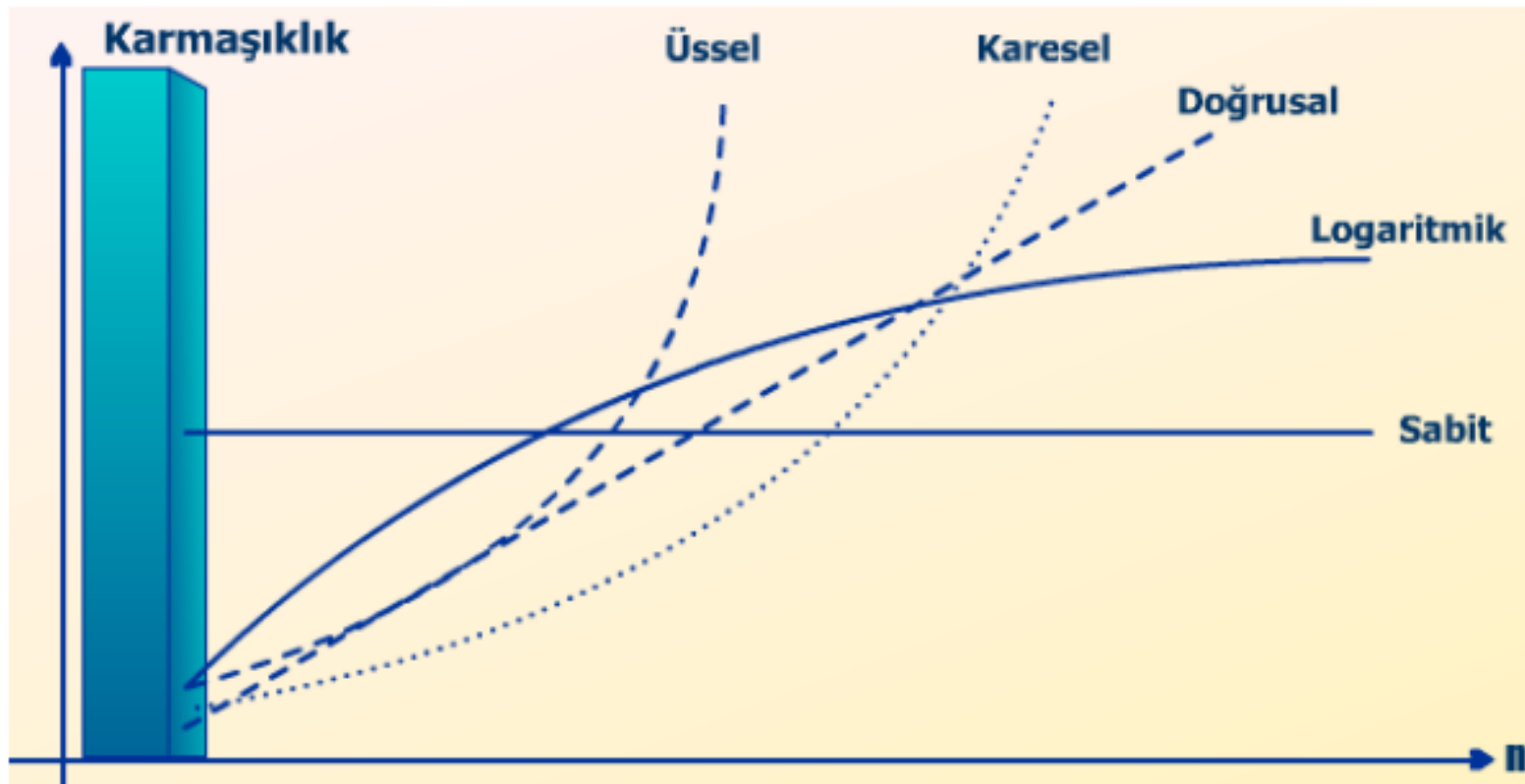
Karmaşıklığı ifade edebilmek için asimtotik ifadeler kullanılmaktadır.

Bu amaçla $O(n)$ (O notasyonu), $\Omega(n)$ (Omega notasyonu), $\theta(n)$ (Teta notasyonu) gibi tanımlamalara baş vurulur.

Karşılaşılan Genel Fonksiyonlar

Maliyet artar	Büyük O	Değişim Şekli
	$O(1)$	Sabit, komut bir veya birkaç kez çalıştırılır. Yenilmez!
	$O(\log_n)$	Logaritmik, Büyük bir problem, her bir adımda sabit kesirler tarafından orijinal problemin daha küçük parçalara ayrılması ile çözülür. İyi hazırlanmış arama algoritmalarının tipik zamanı
	$O(n)$	Doğrusal, Küçük problemlerde her bir eleman için yapılır. Hızlı bir algoritmadır. N tane veriyi girmek için gereken zaman.
	$O(n \log_n)$	Doğrusal çarpanlı logaritmik. Çoğu sıralama algoritması
	$O(n^2)$	Karasel. Veri miktarı az olduğu zamanlarda uygun ($N < 1000$)
	$O(n^3)$	Kübik. Veri miktarı az olduğu zamanlarda uygun ($N < 1000$)
	$O(2^n)$	İki tabanında üssel. Veri miktarı çok az olduğunda uygun ($N \leq 20$)
	$O(10^n)$	On tabanında üssel
	$O(n!)$	Faktöriyel

Karmaşıklık (Complexity)



Algoritma Karmaşıklığı: Büyük-O Notasyonu

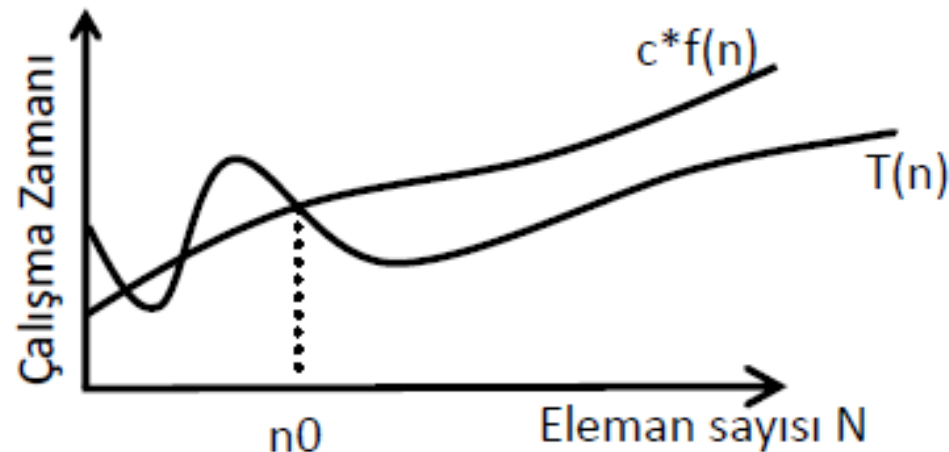
Bir algoritmanın çalışma süresi,

$T(N) = O(f(n))$ O , bir fonksiyon değil, sadece gösterimdir.

$T(N) \leq c f(n)$ ve $N \geq n_0$ koşullarını sağlayan c ve n_0 değerleri varsa $T(N) \leq c f(n)$ ifadesi doğrudur.

$f(n)$, $T(N)$ 'in asimtotik üst limiti olarak adlandırılır.

$T(N) = O(f(n))$



Algoritma Karmaşıklığı: Büyük-O Notasyonu

A algoritması için $T_A(N) = 1000N$ olarak verilmiştir. O notasyonu cinsinden bu çalışma süresi $T_A(N) = 1000N = O(N)$ dir.

Yukarıdaki ifadenin doğruluğunu ispat etmek için O notasyonunun tanımını kullanacağız. Bu tanıma göre $1000N = O(N)$ ifadesini ispatlamak için,

$1000N \leq cN \quad \forall N \geq n_0$ eşitsizliğini belirli bir c ve n_0 sabitleri için ispatlamak gerekir.

$c = 2000$ ve $n_0 = 1$ seçildiği zaman eşitsizliğin n_0 dan büyük her N değeri için sağlandığı aşikardır.

Algoritma Karmaşıklığı: Büyük-O Notasyonu

B algoritması için $T_B(N) = N^2$ olarak verilmiştir. O notasyonu cinsinden bu çalışma süresi $T_B(N) = N^2 = O(N^2)$ dir.

Yukarıdaki ifadenin doğruluğunu ispat etmek için O notasyonunun tanımını kullanacağız. Bu tanıma göre $N^2 = O(N^2)$ ifadesini ispatlamak için,

$N^2 \leq cN^2 \quad \forall N \geq n_0$ eşitsizliğini belirli bir c ve n_0 sabitleri için ispatlamak gerekir.

$c = 1$ ve $n_0 = 1$ seçildiği zaman eşitsizlik n_0 dan büyük her N değeri için sağlanmaktadır.

O Notasyonunun Önemi

İki algoritma karşılaştırılırken zaman mertebesinde konuşulur. Mertebesi büyük olanın daha yavaş olduğu kolaylıkla anlaşılır.

Makineler arasındaki fark, katsayılar olarak düşünüldüğünde $O(7n^2)$ yerine $O(n^2)$ ifadesi geçerli olur. Dolayısıyla sabitler ihmal edilebilir ve birimlerden kurtulur

O Notasyonu-Örnek

- 3n²+2n+5 = O(n²) ifadesinin doğru olup olmadığını ispatlayınız.

$$\begin{aligned} 10n^2 &= 3n^2 + 2n^2 + 5n^2 \\ &\geq 3n^2 + 2n + 5 \text{ için } n \geq 1 \\ c &= 10, n_0 = 1 \end{aligned}$$

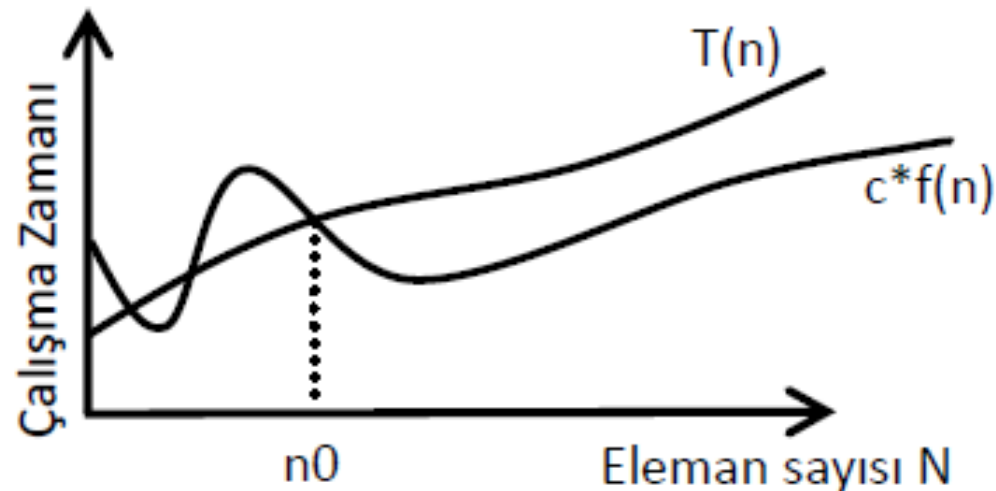
Çözüm kümesini sağlayan kaç tane n_0 ve c çifti olduğu önemli değildir. Tek bir çift olması notasyonun doğruluğu için yeterlidir.

Ω Notasyonu- Asimtotik Alt Limit

— Ω notasyonun tam tersidir.

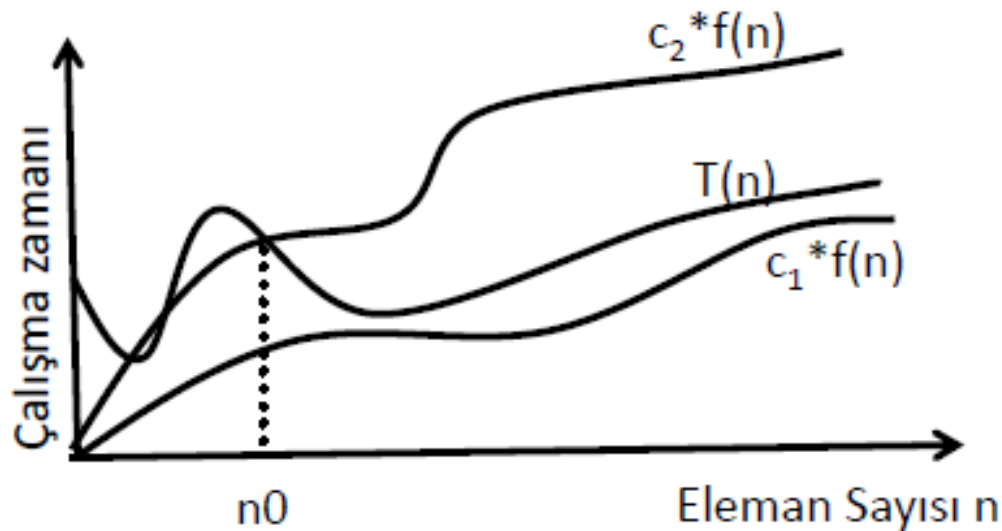
Her durumda $T(N) \geq c f(n)$ ve $N \geq n_0$ koşullarını sağlayan pozitif, sabit c ve n_0 değerleri bulunabiliyorsa $T(N)=\Omega(f(n))$ ifadesi doğrudur.

$f(n)$, $T(N)$ 'in asimtotik alt limiti olarak adlandırılır.



Θ Notasyonu (Ortalama durum analizi)

Her durumda $c_1 f(n) \geq T(n) \geq c_2 f(n)$ ve $n \geq n_0$ koşullarını sağlayan pozitif, sabit c_1, c_2 ve n_0 değerleri bulunabiliyorsa $T(n) = \Theta(f(n))$ ifadesi doğrudur.



T(N) ve O(f(N)) Örnekleri

Çalışma süresi $T(N)$	Asimptotik Karmaşıklık $O(f(N))$
$4 N \log N$	$O(N \log N)$
$7N^2 \log N + 5N^2 + N$	$O(N^2 \log N)$
$10 N^3 + N^2 + 40N + 800$	$O(N^3)$
$2^N + 1000N$	$O(2^N)$

Analiz Kuralları

Aşağıdaki işlemler sabit çalışma süresine sahiptir.

$O(1)$: – Aritmetik (topla, çıkar, çarp, vb.)

- Veri hareketi (oku, yaz, ata)
- Kontrol (dallanma, alt program çağrısı, return)

Döngüler sabit olmayan çalışma süresine sahiptir, $O(N)$, $O(N^2)$, vb.

- Döngü sayısı ile, döngü içindeki işlemlerin toplam çalışma süresi çarpılır.

Ardışık komutlar

- Ardışık olarak birçok komut varsa, çalışma süresi en yüksek maliyete sahip olan komutlar $O()$ 'yu belirler.

Sadeleştirme Kuralları

Mümkünse sadece bir terime indirgeyin.

- En hızlı büyüyen terim, toplam çalışma süresini belirleyicidir.
- N sayısı arttıkça, asimptotik sonuç tamamen derecesi en büyük olan terim tarafından belirlenir.

Sabit katsayılar atılır.

$$O(aN^k + bN^{k-1} + \dots + yN + z) \approx O(N^k)$$

$$O(N) + O(N) = 2 O(N) \approx O(N)$$

$$O(N) + O(M) \approx O(N) \quad \text{Eğer } N > M$$

$$O(N) * O(N) = O(N^2)$$

$$O(N) + O(\log N) \approx O(N) \quad \text{Çünkü } O(N) > O(\log N)$$

$$O(N) * O(\log N) \approx O(N \log N)$$

Matematik İfadeler (1)

- **Üsler**

$$X^A X^B = X^{A+B}$$

$$X^A / X^B = X^{A-B}$$

$$(X^A)^B = X^{AB}$$

$$X^N + X^N = 2X^N$$

$$2^N + 2^N = 2^{N+1}$$

- **Logaritmalar**

Tanım: $X^A = B$ ise $\log_X B = A$

$$\log AB = \log A + \log B$$

$$\log A/B = \log A - \log B$$

$$\log (A^B) = B \log A$$

Matematik ifadeler (2)

- Seriler

$$\sum_{i=1}^N i = \frac{N(N+1)}{2} \cong \frac{N^2}{2}$$

$$\sum_{i=1}^N i^2 = \frac{N(N+1)(2N+1)}{6} \cong \frac{N^3}{3}$$

$$\sum_{i=0}^N 2^i = 2^{N+1} - 1$$

$$\sum_{i=0}^N a^i = \frac{a^{N+1} - 1}{a - 1}$$

Çalışma süresi örnekleri

N	Sabit $O(1)$	Logaritmik $O(\log N)$	Doğrusal $O(N)$	Kare $O(N^2)$
1000	10 ns	100 ns	10 μ s	10 ms
1 milyon	10 ns	200 ns	10 ms	3 saat
1 milyar	10 ns	300 ns	10 sec	300 yıl

1 saniye = 10^3 mili san = 10^6 mikro san = 10^9 nano san

Bazı Veri Yapısı İşlemlerinin Algoritma Karmaşıklığı

	Ekleme	Silme	Arama
Sırasız dizi	$O(1)$	$O(n)$	$O(n)$
Sıralı dizi	$O(\log_2 n + n)$	$O(\log_2 n + n)$	$O(\log_2 n)$
Bağlantılı Liste	$O(n)$	$O(n)$	$O(n)$
İkili Arama Ağacı	$O(\log_2 n)$	$O(\log_2 n)$	$O(\log_2 n)$

Örnek1: Tek döngü

- myfunc() fonksiyonu kaç defa çağrılır?

```
for (i = 1; i <= N; i++)  
{  
    myfunc( );  
}
```

$$\sum_{i=1}^N 1 = N$$

Örnek2: Tek döngü

```
for (i = 1; i <= N; i *= 2)
{
    myfunc( );
}
```

$$\sum_{i=1,2,4,8,16,..}^N 1 = \log_2 N$$

Karmaşıklık $O(\log_2 N)$

Örnek3: İç içe iki döngü

```
for (i = 1; i <= N; i++)  
{  
    for (j = 1; j <= M; j++)  
    {  
        myfunc( );  
    }  
}
```

$$\sum_{i=1}^N \sum_{j=1}^M 1 = \sum_{i=1}^N M = NM$$

Karmaşıklık $O(NM) \approx O(N^2)$

Örnek4: İç içe iki döngü

```
for (i = 1; i <= N; i++)  
{  
    for (j = 1; j <= i; j++)  
    {  
        myfunc( );  
    }  
}
```

$$\sum_{i=1}^N \sum_{j=1}^i 1 = \sum_{i=1}^N i = 1 + 2 + 3 + \dots + N = \frac{N(N+1)}{2}$$

Karmaşıklık $O(N^2/2 + N/2) \approx O(N^2)$

Örnek5: İç içe üç döngü

```
for (i = 1; i <= N; i++) {  
    for (j = 1; j <= M; j++) {  
        for (k = 1; k <= L; k++) {  
            myfunc( );  
        }  
    }  
}
```

$$\sum_{i=1}^N \sum_{j=1}^M \sum_{k=1}^L 1 = \sum_{i=1}^N \sum_{j=1}^M L = \sum_{i=1}^N ML = NML$$

Karmaşıklık $O(NML) \approx O(N^3)$

Örnek6: Ard arda iki döngü

```
for (i = 1; i <= N; i++)
```

```
{
```

```
    myfunc( );
```

```
}
```

```
for (i = 1; i <= N; i *= 2)
```

```
{
```

```
    myfunc( );
```

```
}
```

N

$\log_2 N$

Karmaşıklık $O(N + \log_2 N) \approx O(N)$

Örnek7: Ard arda ve iç içe döngüler

```
for (i = 1; i <= N; i++) {  
    for (j = 1; j <= i; j++) {  
        myfunc( );  
    }  
}  
for (i = 1; i <= N; i++) {  
    for (j = 1; j <= M; j++) {  
        myfunc( );  
    }  
}
```

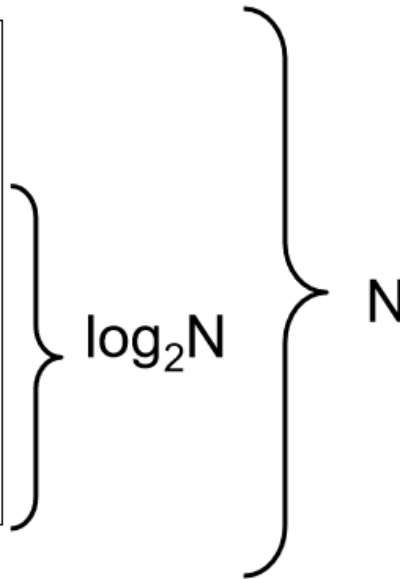
$N(N + 1)/2$

NM

Karmaşıklık $O(N^2/2 + N/2) + O(N^2) \approx 2 O(N^2) \approx O(N^2)$

Örnek8: İç içe iki döngü

```
for (i = 1; i <= N; i++)  
{  
    for (j = 1; j <= M; j *= 2)  
    {  
        myfunc( );  
    }  
}
```



$$\sum_{i=1}^N \sum_{j=1,2,4,\dots}^M 1 = \sum_{i=1}^N \log_2 M = N \log_2 M$$

Karmaşıklık $O(N \log_2 M) \approx N O(\log N)$

Örnek9: İç içe iki döngü

```
for (i = 1; i <= N * N; i++)  
{  
    for (j = 1; j <= N * N * N; j++)  
    {  
        myfunc( );  
    }  
}
```

N^3
 N^2

$$\sum_{i=1}^{N^2} \sum_{j=1}^{N^3} 1 = \sum_{i=1}^{N^2} N^3 = N^2 N^3 = N^5$$

Karmaşıklık $O(N^5)$

Örnek10: Faktoriyel

// İteratif çözüm

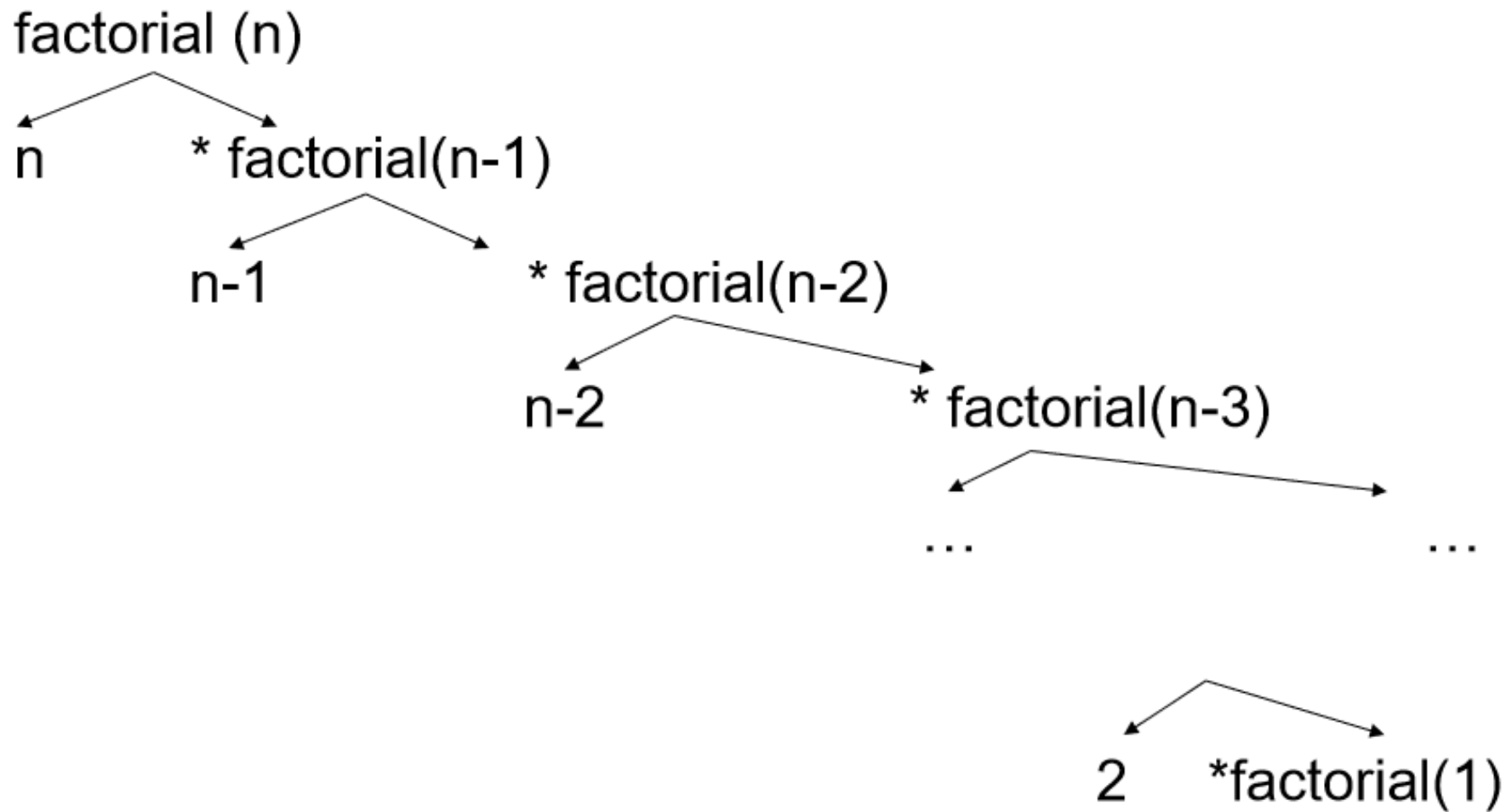
```
int factorial(int n)
{
    int i, fact;
    if (n<=1) return 1;
    else
    {
        fact = 1;
        for (i=2;i<=n;i++)
            fact *= i;
        return fact;
    }
}
```

// Rekürsif çözüm

```
int factorial (int n)
{
    if (n<=1)
        return 1;
    else
        return n * factorial(n-1);
}
```

Her ikisi için karmaşıklık $O(N)$

Rekürsif Faktoriyel



Rekürsif Faktoriyel

- $T(n)$ rekürsif olarak $T(k)$ cinsinden tanımlanır, $k < n$
- Rekürsiyon, $T(n)$ 'nin temel durumlara ($T(0)$ veya $T(1)$) biriktirilmesini sağlar.
- $$\begin{aligned} T(n) &= T(n-1) + d \\ &= T(n-2) + d + d \\ &= T(n-3) + d + d + d \\ &= \dots \\ &= T(1) + (n-1)*d \\ &= c + (n-1)*d \\ &= c + nd - d \\ &\approx O(n) \end{aligned}$$

Örnek11: Rekürsif bölme

```
void divide(int N)
{
    if (N <= 2)
        return;
    divide(N / 2);
}
```

Karmaşıklık $O(\log N)$

$$T(0) = T(1) = T(2) = 1$$
$$T(n) = 1 + T(n/2) \text{ if } n > 2$$

$$\begin{aligned} T(n) &= 1 + (1 + T(n/4)) \\ &= 2 + T(n/4) \\ &= 2 + (1 + T(n/8)) \\ &= 3 + T(n/8) \\ &= 3 + (1 + T(n/16)) \\ &= 4 + T(n/16) \\ &\dots \\ &\approx O(\log_2 n) \end{aligned}$$

k adım sonra:

$$T(n) = k + T(n/2^k)$$

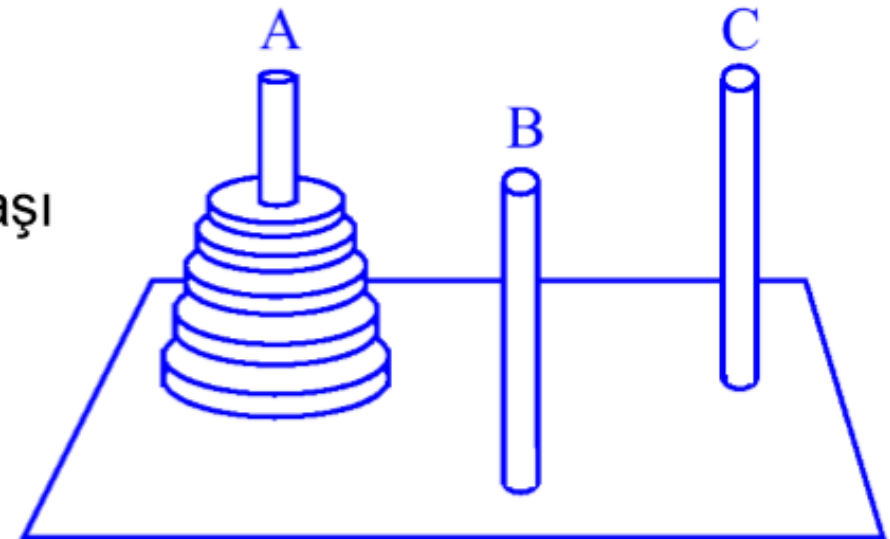
Rekürsiyon,
olduğunda $n/2^k = 1$
biter.

Buradan: $2^k = n$

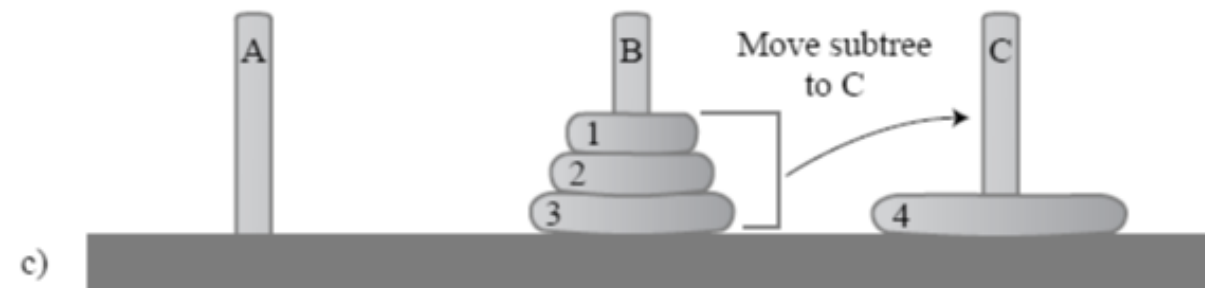
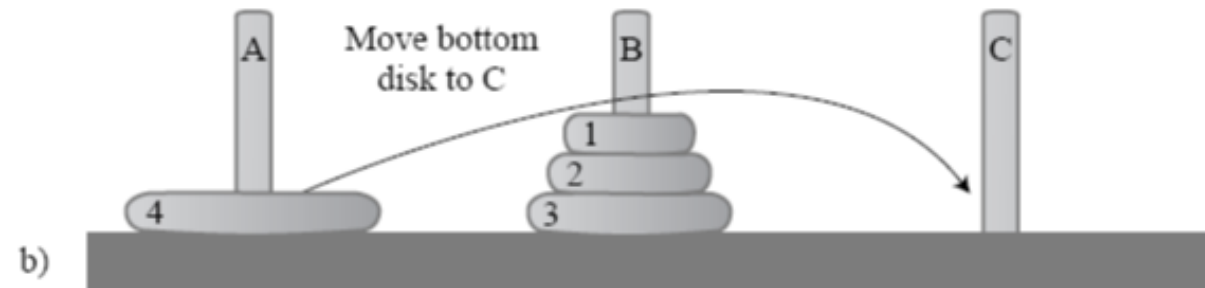
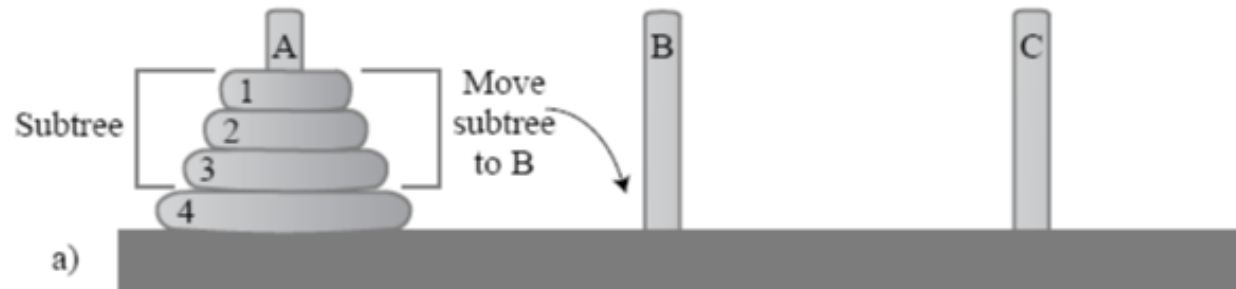
$$k = \log_2 n$$

Örnek12: Hanoi Kuleleri

- **Amaç:** *N adet* disk A kulesinden C kulesine taşımak.
- **Kurallar:**
 - Bir defada tek bir disk taşınır
 - Büyük bir disk, küçük bir diskin üzerine gelemmez
- **Rekürsif çözüm:**
 - *N - 1* disk A'dan B'ye taşı
 - A'daki en büyük disk (tek) C'ye taşı
 - *N - 1* disk B'den C'ye taşı
- **Toplam hareket sayısı:**
 - $O(2^N)$



Rekürsif çözüm



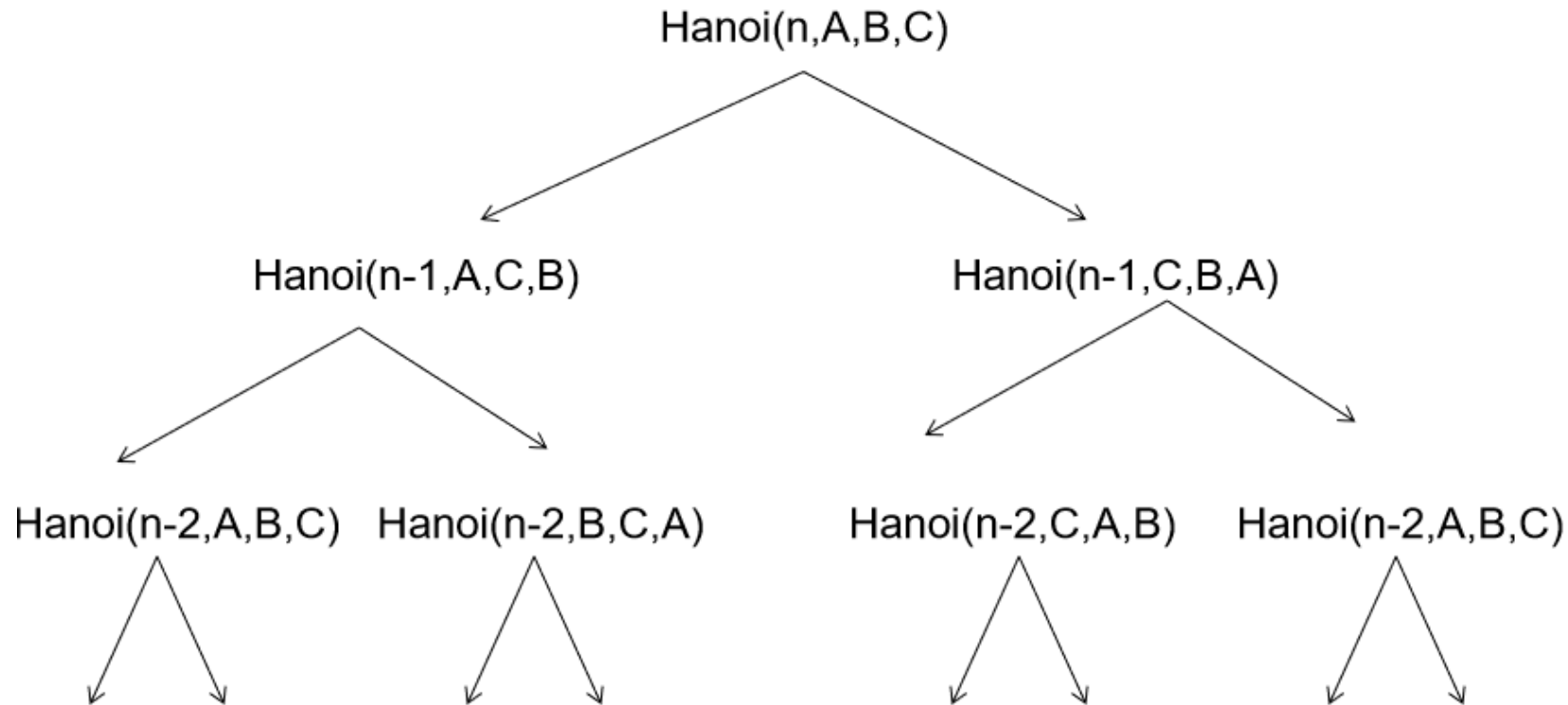
Rekürsif Hanoi Kuleleri

```
void hanoi(int N, char source, char target, char tmp)
{
    if (N == 1)
        printf("Move disk from %c to %c \n", source, target);
    else {
        hanoi(N-1, source, tmp, target);
        printf("Move disk from %c to %c \n", source, target);
        hanoi(N-1, tmp, target, source);
    }
}

int main()
{
    move(4, 'A', 'B', 'C');
    return 0;
}
```

Karmaşıklık $O(2^N)$

Rekürsif Hanoi Kuleleri



$$\begin{aligned} T(N) &= 2 * T(N-1) \\ &= 2 * (2 * T(N-2)) \\ &= 2 * (2 * (2 * T(N-3))) \\ &= \dots \\ &= 2^{N-1} * T(1) \\ &\approx O(2^N) \end{aligned}$$

Rekürsif Hanoi Kuleleri

- Rekürsion bağıntısı:

$$T(n) = 2 T(n - 1) + 1$$

$$T(1) = 1$$

- Bağıntıyı açarak çözüm:

$$T(n) = 2 (2 T(n - 2) + 1) + 1 =$$

$$= 4 T(n - 2) + 2 + 1 =$$

$$= 4 (2 T(n - 3) + 1) + 2 + 1 =$$

$$= 8 T(n - 3) + 4 + 2 + 1 = \dots$$

$$= 2^i T(n - i) + 2^{i-1} + 2^{i-2} + \dots + 2^1 + 2^0$$

- $i = n - 1$ olduğunda açılım sonlanır.

$$T(n) = 2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 2^1 + 2^0$$

- Sonuç geometrik toplamdır.

- $T(N) = 2^N - 1 = O(2^N)$

- Bu algoritma üslüdür.

Örnek13: Dizi sıralama

```
/* Sort an array in ascending order */  
void BubbleSort(int a[ ], int N) {  
    int pass, i, hold;  
  
    /* loop to control number of passes */  
    for ( pass = 1; pass < N; pass++ ) {  
        /* loop to control number of comparisons per pass */  
        for ( i = 0; i < N - 1; i++ ) {  
            /* compare adjacent elements and swap them if first  
            element is greater than second element */  
            if ( a[ i ] > a[ i + 1 ] ) {  
                hold = a[ i ];  
                a[ i ] = a[ i + 1 ];  
                a[ i + 1 ] = hold;  
            } /* end if */  
        } /* end inner for */  
    } /* end outer for */  
}
```

Karmaşıklık $O(N^2)$

Başlangıç

40
10
90
30
20
60

Sıralı dizi

10
20
30
40
60
90

Dizi sıralama

- **En iyi durum:** Sıralama öncesi elemanlar zaten sıralı ise.
Çalışma süresi doğrusaldır. $O(N)$

- **En kötü durum:** Sıralama öncesi tüm elemanlar tamamen ters sıralı ise.
Çalışma süresi kareseldir. $O(N^2)$

- **Ortalama durum:** Sıralama öncesi elemanlar kısmen sıralı, kısmen sırasız iseise.

■ Çalışma süresi kareseldir. $O(N^2/2) \approx O(N^2)$

Program çalışma süresinin C'de ölçülmesi

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    clock_t baslama;
    double sure, i;

    baslama = clock(); // Capture the start time

    for (i=1; i <= 1E5; i++)
        printf("Sayac = %g \r", i);

    sure = (clock() - baslama) / (double) CLOCKS_PER_SEC;
    printf("Geçen süre : %g saniye \n", sure);

    system("pause");
    return 0;
}
```

Örnek14: Dizide İkili Arama

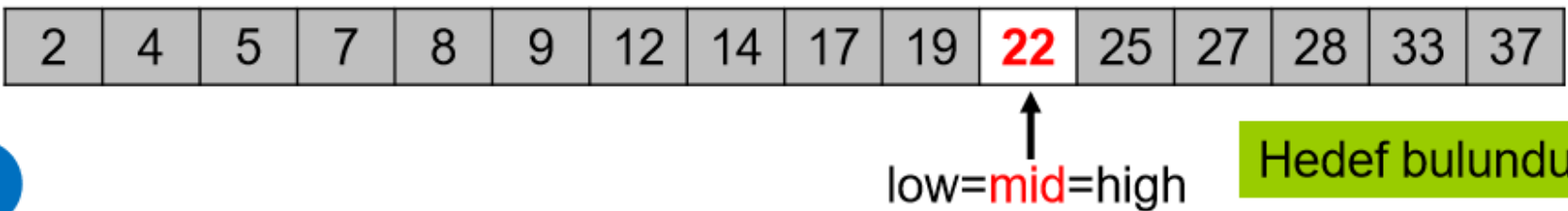
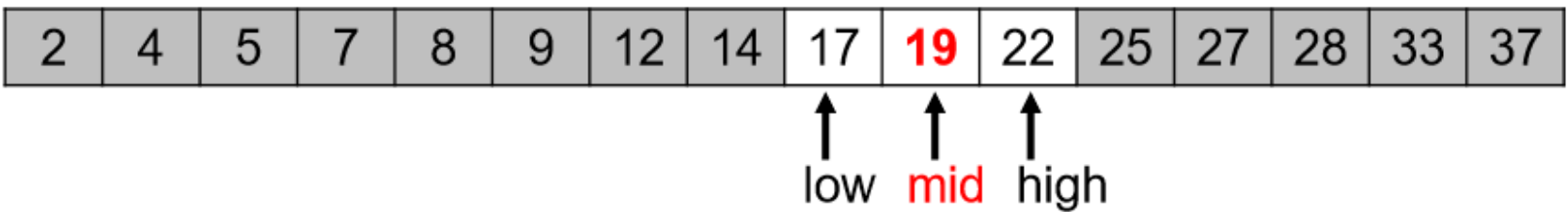
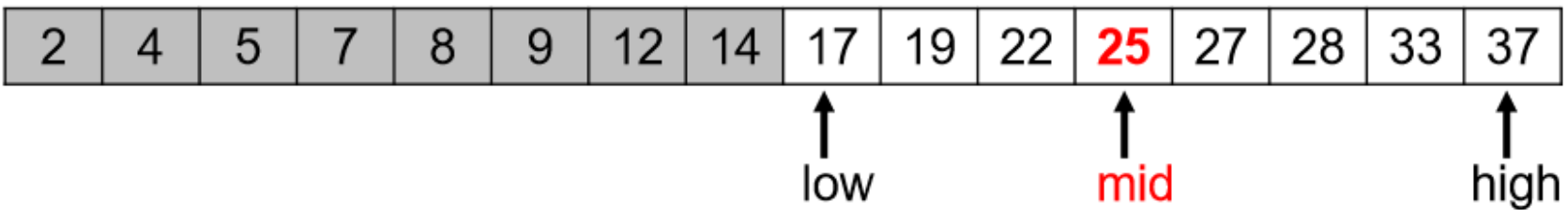
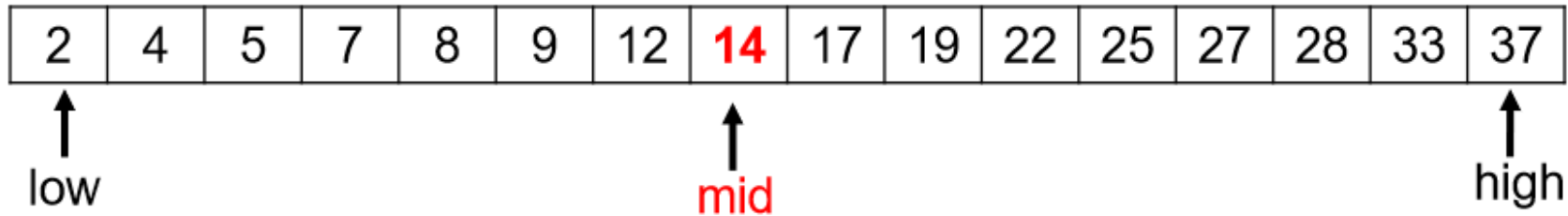
İkili arama

- Sadece sıralı bir dizide uygulanabilir.
- Bu nedenle, dizi arama işlemi öncesinde sıralı hale getirilmelidir.
- Ortadaki elemanı arama anahtarı ile karşılaştır
 - Eşitseler, aranan bulundu
 - Eğer $key < middle$ ise, dizinin ilk yarısında aramaya devam et.
 - Eğer $key > middle$ ise, dizinin ikinci yarısında aramaya devam et.
 - Yeni $middle$ 'i hesapla ve üstteki adımları tekrarla.
- Çok hızlı; en fazla x adımda çözüm bulunur. ($2^x > \text{Eleman sayısı}$)
 - Örnek: 30 elemanlı dizide arama işlemi en fazla 5 adımda biter.
($2^5 > 30$)

Karmaşıklık $O(\log_2 N)$

İkili Arama örneği:

Aranan hedef = 22



Ders Sonu

Sorular